机器学习：从直观例子到公式推导

天国　付

2026 年 11 月

摘要

本笔记按照循序渐进的五课组织：机器学习基本概念、线性回归与最小二乘、梯度下降、逻辑回归、模型评估与泛化。先用简短的数据和手工计算解释每个新概念，再给出一般公式及必要的适用条件，并配合可运行的 Python 代码验证。复杂矩阵表达式与优化理论作为选读内容，不代替入门所需的理解过程。

关键词：机器学习；均方误差；线性回归；梯度下降；逻辑回归；泛化能力

目录

1　阅读说明：这份笔记怎样使用？　4

2　第一课：从数据理解机器学习　4

2.1　先从一个真实会遇到的问题开始 4
2.2　什么是样本、特征与标记? 4
2.3　监督学习、无监督学习、回归与分类 5
2.4　模型、参数、策略、算法不是同一回事 5
2.5　预测错了怎么办？先理解损失函数 6
2.6　用最简单的算例完成一次“训练” 6
2.7　再认识通用写法：经验风险 7
2.8　训练误差为零，仍可能预测不好 7
2.9　这一课应当掌握什么 7

3　第二课：线性回归与最小二乘法　8

3.1　为什么需要拟合一条直线? 8
3.2　模型中的 w 与 b 是什么? 8
3.3　从残差一步一步走到 MSE 8
3.4　为什么教材有时把分母写成 $2n$? 9
3.5　第一步：对截距 b 求偏导 9
3.6　第二步：对斜率 w 求偏导 9
3.7　第三步：把四组数据真正算出来 10
3.8　为什么这次偏导为零可以找到最小值? 10

3.9 选读：多特征时的矩阵公式 . . . 11
3.10 这一课应当掌握什么 . . . 11

4 第三课：用导数一步一步找到更好的参数 11
4.1 为什么需要梯度下降？ . . . 11
4.2 先从一个一元二次函数开始 . . . 12
4.3 完整手算四次迭代 . . . 12
4.4 为什么“导数”会变成“梯度”？ . . . 12
4.5 线性回归：从损失函数求出两个偏导 . . . 13
4.6 手工训练一个两样本模型 . . . 13
4.7 怎么理解学习率？ . . . 14
4.8 选读：一般参数的统一记号 . . . 14
4.9 这一课应当掌握什么 . . . 14

5 第四课：逻辑回归与二分类 15
5.1 成绩是数值，“能否通过”却是类别 . . . 15
5.2 Sigmoid 函数怎么把实数变成概率？ . . . 15
5.3 把线性模型代进去，就得到逻辑回归 . . . 15
5.4 为什么叫“对数几率回归”？（理解即可） . . . 16
5.5 概率有了，参数怎么训练？ . . . 16
5.6 交叉熵为什么合理？先看几个数 . . . 17
5.7 从求导得到梯度更新公式 . . . 17
5.8 手算一次逻辑回归训练 . . . 18
5.9 选读：凸性、可分性与边界条件 . . . 18
5.10 这一课应当掌握什么 . . . 18

6 第五课：过拟合、泛化与模型评价 19
6.1 训练数据全预测对了，模型就是好模型吗？ . . . 19
6.2 什么是过拟合和欠拟合？ . . . 19
6.3 训练误差、测试误差、泛化误差有什么区别？ . . . 20
6.4 如何划分训练集、验证集和测试集？ . . . 20
6.5 留出法与五折交叉验证 . . . 20
6.6 容易忽略的数据泄漏 . . . 21
6.7 偏差与方差：先理解含义，再看公式 . . . 21
6.8 回归模型怎么评价？ . . . 22
6.9 分类模型怎么评价？先读懂混淆矩阵 . . . 22
6.10 一次多项式实验为什么有帮助？ . . . 23
6.11 这一课应当掌握什么 . . . 23

7 动手验证：把公式翻译成 Python 23

7.1 实验一：用网格搜索找 w 24
7.2 实验二：最小二乘公式与两次求偏导 24
7.3 实验三：反复迭代就是梯度下降 25
7.4 实验四：逻辑回归的一步参数更新 26
7.5 实验五：训练误差下降为何不代表测试误差下降？ 27
7.6 实验记录应当保存什么？ 27

8 选读：下一阶段的学习路线 **27**
8.1 正则化：除了拟合，还要控制复杂度 27
8.2 从一个变量到很多变量 27
8.3 从训练模型到理解统计不确定性 27
8.4 阅读前的三个问题 27

A 常用符号：先查含义，再读公式 **28**

B 修订记录 **28**

参考文献 **30**

1 阅读说明：这份笔记怎样使用？

这份机器学习笔记从可手算的例子出发：先观察数据和预测，再引入损失函数、参数估计与迭代算法，最后讨论模型在新数据上的表现。每当出现抽象公式，先解释符号，再用具体数值计算，最后才写一般形式。这样可以把统计学、数学分析和 Python 实践连接起来。

第一课以西瓜分类与学习时间预测为例，认识样本、特征、标记、模型、参数和损失；第二课从四组数据推导线性回归的最小二乘解；第三课从 $f(x)=(x-3)^2$ 学会梯度下降，再训练两个参数；第四课使用 Sigmoid、伯努利似然和交叉熵构造二分类模型；第五课比较训练误差、测试误差和泛化误差，学习交叉验证、数据泄漏和分类指标。

阅读建议：第一次阅读重点看每节的“为什么”和手工算例；第二次自己推一遍公式、核对样本表格；第三次运行配套 Python 脚本。标有“选读”的矩阵公式或优化边界可以暂时跳过，不影响掌握前五课的主线。

资料主要参考周志华《机器学习》、李航《统计学习方法（第 2 版）》、James 等人的 *An Introduction to Statistical Learning: With Applications in Python* 和基础最优化教材[1-4]。

2 第一课：从数据理解机器学习

2.1 先从一个真实会遇到的问题开始

如果需要判断一个西瓜是不是好瓜，最直接的办法是提前写出规则，例如“色泽青绿且敲声浊响就判为好瓜”。但规则是谁给计算机的？是程序员。现实情况可能远比一两条规则复杂：同样的颜色可能对应不同结果，根蒂、纹理等信息也会影响判断。

另一种方法是先收集已经知道结果的样本，让程序从这些样本中建立预测规则。机器学习研究的正是如何利用已有数据，得到能对新数据进行预测或分析的模型。这里的关键不是“让计算机记住答案”，而是希望它学到能够用于未见数据的规律。

表 1: 为了说明概念而构造的西瓜样本（不是实际数据）

色泽	根蒂	敲声	已知结果
青绿	蜷缩	浊响	好瓜
乌黑	蜷缩	浊响	好瓜
浅白	硬挺	清脆	坏瓜
青绿	硬挺	清脆	坏瓜

2.2 什么是样本、特征与标记？

先换成更容易用数学表达的例子：根据每天的学习时间预测成绩。假设记录了四名学生：

学生	学习时间 x（小时）	成绩 y（分）
A	1	60
B	2	70
C	3	80
D	4	90

每一行叫作一个**样本**；学习时间是用于预测的**特征**（输入 x）；已知成绩是**标记**或响应变量（输出 y）。

若有 m 个样本，就可将训练数据写为

$$\boxed{D = \{(x_i, y_i)\}_{i=1}^{m}.} \tag{2.1}$$

i 表示第几个样本，而非指数。如果每个样本包含 d 个数值特征，如学习时间、出勤率和作业得分，则输入变成列向量

$$\boldsymbol{x}_i = (x_{i1}, x_{i2}, \ldots, x_{id})^{\mathsf{T}} \in \mathbb{R}^d.$$

先理解单个 x，再理解向量 $\boldsymbol{x}$。西瓜颜色等类别型特征不能直接当作任意实数，需要适当编码，本课暂不展开。

2.3 监督学习、无监督学习、回归与分类

监督学习的训练样本提供了待预测的正确结果。例如已有学习时间和真实成绩，便可以训练成绩预测模型。**无监督学习**通常没有这样的目标标记，算法可能根据学习时间与娱乐时间等信息把学生分组；聚类就是一种典型任务。所谓“无监督”不是“没有数据”，而是没有提供任务对应的目标标记。

监督学习常见两种任务：

任务	预测目标	例子
回归（Regression）	连续数值	房价、气温、成绩
分类（Classification）	离散类别	邮件是否为垃圾邮件

这是按**预测目标**划分任务，而不是按程序使用了哪种求解器划分。

2.4 模型、参数、策略、算法不是同一回事

假设先使用一条过原点的直线预测：

$$\boxed{\hat{y} = wx.} \tag{2.2}$$

$\hat{y}$（读作“y 帽”）表示预测值；x 是输入；w 是**模型参数**，决定这条直线的斜率。若 $w = 5$，则学习两小时预测 10 分；若 $w = 10$，则预测 20 分。改变 w，模型的预测规则就变了。

现在允许直线有截距：

$$\hat{y} = wx + b, \qquad w, b \in \mathbb{R}.$$

所有这种形式的函数构成一个**模型族**（或假设空间）：

$$\mathcal{F} = \{f_{w,b}(x) = wx + b \mid w, b \in \mathbb{R}\}. \tag{2.3}$$

这里的竖线 | 读作“满足右边条件的”；$w, b \in \mathbb{R}$ 表示两个参数可以取实数。**模型族**告诉我们允许使用什么形式的函数；**参数**决定最终是哪一条直线；**算法**负责从数据中寻找合适参数。

李航《统计学习方法》常用“模型—策略—算法”概括这件事：模型规定学什么，策略规定什么算好，算法规定怎么求出来[2]第 1 章。

2.5 预测错了怎么办？先理解损失函数

为了便于心算，再用三组人工数据：

$$(x_1, y_1) = (1, 2), \quad (x_2, y_2) = (2, 4), \quad (x_3, y_3) = (3, 6).$$

采用模型 $\hat{y} = wx$，如果先猜 $w = 1$，预测就是 $(1, 2, 3)$，而真实结果为 $(2, 4, 6)$。我们需要衡量预测与真实值之间的差别。

单个样本的预测误差可写成 $e_i = \hat{y}_i - y_i$；仅将误差直接相加可能发生正负抵消，因此常把误差平方，定义**平方损失**

$$\boxed{L(y_i, \hat{y}_i) = (y_i - \hat{y}_i)^2.} \tag{2.4}$$

例如真实值为 4、预测值为 2，损失为 $(4-2)^2 = 4$；如果预测改为 3，损失降为 1。平方使较大的误差受到更大惩罚，但也意味着这种方法对离群值可能较敏感。

将所有样本的平方损失求平均，得到**均方误差**（MSE）：

$$\boxed{\mathrm{MSE}(w) = \frac{1}{m} \sum_{i=1}^{m} (wx_i - y_i)^2.} \tag{2.5}$$

辨别三个量： L 衡量一个样本；MSE 衡量这批样本的平均平方误差；w 是使误差发生变化、等待求解的参数。

2.6 用最简单的算例完成一次“训练”

把三个样本代入，设 $J(w) = \mathrm{MSE}(w)$，便有

$$J(w) = \frac{1}{3}\left[(w-2)^2 + (2w-4)^2 + (3w-6)^2\right] \tag{2.6}$$

$$= \frac{1}{3}(1 + 4 + 9)(w-2)^2 \tag{2.7}$$

$$= \frac{14}{3}(w-2)^2. \tag{2.8}$$

平方永远非负，所以当且仅当 $w = 2$ 时，J 最小且等于 0：

$$\boxed{w^* = 2, \qquad J(w^*) = 0, \qquad \hat{y} = 2x.} \tag{2.9}$$

这已经是一个完整的训练过程：**提供数据** → **选择模型** → **定义损失** → **确定参数**。本题靠完全平方便可求解，不需要使用梯度下降。

如果用优化语言，刚才求解的是

$$\min_{w \in \mathbb{R}} J(w).$$

$\min J$ 指的是最小函数值；$\arg\min J$ 指的是让函数取得最小值的参数。在这里，最小值是 0，而使它最小的参数是 $w = 2$。要注意区分。

2.7　再认识通用写法：经验风险

上面 MSE 是一种具体的损失。若把损失函数统一写为 L，模型统一写为 f，则训练样本上的**经验风险**为

$$\widehat{R}_D(f) = \frac{1}{m}\sum_{i=1}^{m} L(y_i, f(x_i)). \tag{2.10}$$

这条看似复杂的公式，其实只是在说“对每个样本计算损失，再求平均”。在平方损失和 $f_w(x) = wx$ 的情况下，它恰好就是前面算过的 MSE。选择使平均损失最小的模型叫作**经验风险最小化**（ERM）：

$$\hat{f} \in \arg\min_{f \in \mathcal{F}} \widehat{R}_D(f). \tag{2.11}$$

阅读顺序：先看得懂的数值算例，再把具体的 w、MSE 换成一般的 f、L，就容易理解这两个记号。这里并没有强制规定必须使用哪一种求解算法。

2.8　训练误差为零，仍可能预测不好

刚才 $f(x) = 2x$ 完全拟合三条训练记录。但另一个函数

$$g(x) = 2x + (x-1)(x-2)(x-3)$$

也在 $x = 1, 2, 3$ 时分别预测 $2, 4, 6$，因为乘积中总有一个因子为零。

可是面对没有见过的 $x = 4$，两者预测为

$$f(4) = 8, \qquad g(4) = 8 + 3 \times 2 \times 1 = 14.$$

同样的零训练误差，并不意味着相同的未来预测。这就是为什么评估模型时不能只检查训练数据。

如果假设新数据来自某个目标联合分布 $P_{X,Y}$，便可用**总体风险**描述平均预测损失：

$$R(f) = \mathbb{E}_{(X,Y)\sim P_{X,Y}}[L(Y, f(X))]. \tag{2.12}$$

当前不要求直接计算这条式子。$\mathbb{E}$ 表示按目标数据分布求期望；它与把固定训练集上若干损失直接求平均不同。真实数据分布通常未知，因此我们会用独立测试数据估计预测表现。

2.9　这一课应当掌握什么

1. 样本是记录，特征用于预测，标记是已有的目标答案。
2. 监督学习有目标标记；无监督学习通常要从无目标标记的数据中发现结构。
3. 模型规定预测形式，参数决定具体预测，损失函数评判好坏，算法寻找参数。
4. 对单变量模型 $\hat{y} = wx$，能够手算 MSE 并通过完全平方找出最优 w。
5. 训练误差小不保证对新数据预测得好。

自检：真实值 $y = 5$、预测值 $\hat{y} = 3$，平方损失是多少？为什么把所有误差直接相加可能误导我们？

本课概念主要对应周志华《机器学习》第 1 章、李航《统计学习方法》第 1 章，以及 ISLP 第 2 章[1-3]。

3 第二课：线性回归与最小二乘法

3.1 为什么需要拟合一条直线？

上一课的三组数据恰好满足 $y = 2x$，训练误差可以达到零。现实数据往往并不这样整齐。考虑四组为了手算方便而构造的记录：

学生	学习时间 x（小时）	成绩 y（教学数值）
A	1	2
B	2	3
C	3	5
D	4	4

如果用一条直线近似这些点，哪条直线算“最好”？线性回归提供函数形式，最小二乘法提供选择直线参数的准则。这里的成绩是人工数据，并不是实际考试成绩。

3.2 模型中的 w 与 b 是什么？

假设输入和输出之间近似满足

$$\boxed{\hat{y}_i = wx_i + b.} \tag{3.1}$$

x_i 是第 i 个输入，y_i 是真实结果，$\hat{y}_i$ 是预测结果；w 是斜率（权重），b 是截距（偏置）。输入和真实结果属于数据，w, b **才是需要由训练确定的参数**。

例如随手选择 $w = 1, b = 1$，模型就是 $\hat{y} = x + 1$。当 $x = 3$ 时预测为 4，而样本真实结果是 5。这说明需要有一套评价参数好坏的规则。

3.3 从残差一步一步走到 MSE

定义**残差**为

$$e_i = y_i - \hat{y}_i = y_i - (wx_i + b). \tag{3.2}$$

在 $w = 1, b = 1$ 的情况下，四个样本有

x_i	y_i	$\hat{y}_i$	e_i
1	2	2	0
2	3	3	0
3	5	4	1
4	4	5	-1

残差相加得到 $0 + 0 + 1 - 1 = 0$，但模型显然没有预测对全部样本。为了避免正负抵消，将残差平方再求和：

$$\mathrm{RSS}(w, b) = \sum_{i=1}^{n} (y_i - wx_i - b)^2. \tag{3.3}$$

将 RSS 除以样本数 n，就是**均方误差**：

$$\boxed{\mathrm{MSE}(w, b) = \frac{1}{n} \sum_{i=1}^{n} (wx_i + b - y_i)^2.} \tag{3.4}$$

对于这四条数据，当前 MSE $= (0+0+1+1)/4 = 0.5$。最小二乘法要求找到 w, b，使 RSS（等价地，使 MSE）最小。

3.4 为什么教材有时把分母写成 $2n$？

为了计算方便，训练时还常写

$$\boxed{J(w,b) = \frac{1}{2n}\sum_{i=1}^{n}(wx_i + b - y_i)^2.} \tag{3.5}$$

它和 MSE 相比多乘了 $1/2$：MSE $= 2J$。**数值不同，但取到最小值的 w, b 完全相同**。加上 $1/2$ 只是为了消掉平方求导产生的系数 2。不能把 J 的数值直接称为 MSE。

现在要解决的问题可以写成

$$\min_{w,b\in\mathbb{R}} J(w,b).$$

这就是一个二元函数的极小化问题。接下来只用学过的偏导数来求参数。

3.5 第一步：对截距 b 求偏导

求 $\partial J/\partial b$ 时，把 w 和样本数据都看成常数。利用链式法则：

$$\frac{\partial J}{\partial b} = \frac{1}{n}\sum_{i=1}^{n}(wx_i + b - y_i). \tag{3.6}$$

令它等于 0，先展开求和，再两边除以 n：

$$\begin{aligned}\sum_{i=1}^{n}(wx_i + b - y_i) &= 0,\\ w\sum_{i=1}^{n}x_i + nb - \sum_{i=1}^{n}y_i &= 0,\\ w\bar{x} + b - \bar{y} &= 0.\end{aligned}$$

这里

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n}x_i, \qquad \bar{y} = \frac{1}{n}\sum_{i=1}^{n}y_i$$

分别是输入、输出的样本均值。所以

$$\boxed{b = \bar{y} - w\bar{x}.} \tag{3.7}$$

它告诉我们：**带截距项的普通最小二乘直线经过均值点** $(\bar{x}, \bar{y})$。这一性质是求导推出来的，不需要死记。

3.6 第二步：对斜率 w 求偏导

现在把 b 看成常数，有

$$\frac{\partial J}{\partial w} = \frac{1}{n}\sum_{i=1}^{n}x_i(wx_i + b - y_i). \tag{3.8}$$

令偏导为零，并代入刚求得的 $b = \bar{y} - w\bar{x}$：

$$\sum_{i=1}^{n}x_i\big[w(x_i - \bar{x}) - (y_i - \bar{y})\big] = 0.$$

因为 $\sum_i (x_i-\bar{x})=0$、$\sum_i (y_i-\bar{y})=0$，整理得

$$w\sum_{i=1}^{n}(x_i-\bar{x})^2=\sum_{i=1}^{n}(x_i-\bar{x})(y_i-\bar{y}).$$

只要 x_i 不全部相同，分母就大于零，于是得到熟悉的最小二乘公式：

$$\boxed{\hat{w}=\frac{\sum_{i=1}^{n}(x_i-\bar{x})(y_i-\bar{y})}{\sum_{i=1}^{n}(x_i-\bar{x})^2},\qquad \hat{b}=\bar{y}-\hat{w}\bar{x}.} \tag{3.9}$$

读公式的方法：分子衡量 x 与 y 围绕均值一起变动的程度；分母衡量 x 自身的变动程度。统计学中也能写作 $\hat{w}=S_{xy}/S_{xx}$；若样本协方差、方差采用相同的分母约定，则 $\hat{w}=\widehat{\mathrm{Cov}}(X,Y)/\widehat{\mathrm{Var}}(X)$。

3.7 第三步：把四组数据真正算出来

回到开头的样本，先求均值：

$$\bar{x}=\frac{1+2+3+4}{4}=2.5,\qquad \bar{y}=\frac{2+3+5+4}{4}=3.5.$$

然后列一个表核对斜率的分子和分母：

$x_i-\bar{x}$	$y_i-\bar{y}$	两者乘积	$(x_i-\bar{x})^2$
-1.5	-1.5	2.25	2.25
-0.5	-0.5	0.25	0.25
0.5	1.5	0.75	0.25
1.5	0.5	0.75	2.25
	合计	4	5

因此

$$\hat{w}=\frac{4}{5}=0.8,\qquad \hat{b}=3.5-0.8\times 2.5=1.5.$$

最终拟合直线为

$$\boxed{\hat{y}=0.8x+1.5.} \tag{3.10}$$

在四个训练点上，其预测值分别为 $2.3, 3.1, 3.9, 4.7$；平方误差为 $0.09, 0.01, 1.21, 0.49$。故

$$\boxed{\mathrm{MSE}=\frac{0.09+0.01+1.21+0.49}{4}=0.45.}$$

和刚才随手选的 $w=1, b=1$ 时的 0.5 相比，训练误差确实降低了。

若代入新输入 $x=5$，预测 $\hat{y}=0.8\times 5+1.5=5.5$。它是模型给出的数学预测，不是现实成绩；而且 5 超出了训练输入的范围 $[1,4]$，这种预测属于**外推**，需要格外谨慎。

3.8 为什么这次偏导为零可以找到最小值？

对于一般二元函数，偏导数为零只是驻点条件，并不自动意味着极小值。这里不同：$J(w,b)$ 是若干平方项之和，是关于参数的**凸二次函数**。若输入不全相同，含截距的两个参数可以唯一确定，因此上面的解是全局最小点。若所有 x_i 相同，斜率无法被唯一识别。

3.9 选读：多特征时的矩阵公式

以上是初学者最应该掌握的内容。若每个样本有很多特征，可将设计矩阵写成 $X \in \mathbb{R}^{n\times p}$，参数向量写成 $\boldsymbol{\beta} \in \mathbb{R}^p$，输出写成 $\boldsymbol{y} \in \mathbb{R}^n$。为了处理截距，可在 X 中增加一列全 1。

平方损失的矩阵形式为

$$J(\boldsymbol{\beta}) = \frac{1}{2n}\|X\boldsymbol{\beta} - \boldsymbol{y}\|_2^2,$$

对应梯度

$$\nabla J(\boldsymbol{\beta}) = \frac{1}{n}X^{\mathsf{T}}(X\boldsymbol{\beta} - \boldsymbol{y}). \tag{3.11}$$

令梯度为零得到**正规方程**：

$$X^{\mathsf{T}}X\hat{\boldsymbol{\beta}} = X^{\mathsf{T}}\boldsymbol{y}. \tag{3.12}$$

当 X 满列秩时，$X^{\mathsf{T}}X$ 可逆，于是

$$\hat{\boldsymbol{\beta}} = (X^{\mathsf{T}}X)^{-1}X^{\mathsf{T}}\boldsymbol{y}. \tag{3.13}$$

不要把矩阵公式当作这一课必须背诵的起点。先熟悉上面四组数据的两次偏导和手算，再学习矩阵形式会更自然。实际数值计算一般使用稳定的最小二乘求解接口，而不是直接显式求逆。最小二乘本身不要求误差必须服从正态分布；把它进一步解释为高斯模型的极大似然估计时，才会附加相关概率假设。

3.10 这一课应当掌握什么

1. 能区分输入 x_i、真实值 y_i、预测值 $\hat{y}_i$ 与参数 w, b。
2. 会从残差写出 RSS 和 MSE，知道 $J = \text{MSE}/2$ 的用途。
3. 会用偏导数一步一步推得 $\hat{b} = \bar{y} - \hat{w}\bar{x}$ 和斜率公式。
4. 能按表格手算 $\bar{x}, \bar{y}, \hat{w}, \hat{b}$，并计算训练 MSE。
5. 知道最小训练误差不等于对未来数据预测一定准确。

自检：为什么四个残差相加等于零，却不能说明模型预测完全正确？为什么求导时分母可以选 $2n$？

本课对应周志华《机器学习》线性回归部分、ISLP 简单线性回归部分，以及基础最小二乘优化内容[1,3-4]。

4 第三课：用导数一步一步找到更好的参数

4.1 为什么需要梯度下降？

上一课用最小二乘公式直接求出了斜率和截距，但并不是每个机器学习模型都存在方便使用的解析解。遇到复杂的目标函数，可以先任选一个参数初值，计算当前位置的导数，向使目标函数减小的方向迈一小步，再重复这个过程。这就是**梯度下降法**（Gradient Descent）的基本思想。

这一课先不研究 Lipschitz 连续性、收敛速度等复杂条件，只用一元函数求导、二元函数偏导和简单算例理解算法为什么能工作。

4.2　先从一个一元二次函数开始

设

$$f(x) = (x-3)^2. \tag{4.1}$$

显然 $x=3$ 时取得最小值 0。假装我们还不知道答案，只知道求导：

$$f'(x) = 2(x-3).$$

- 当 $x<3$ 时，导数为负，适当**增大** x 有利于减小函数值。
- 当 $x>3$ 时，导数为正，适当**减小** x 有利于减小函数值。

两个情况用一个更新公式统一表示：

$$\boxed{x_{\text{新}} = x_{\text{旧}} - \eta f'(x_{\text{旧}}), \qquad \eta > 0.} \tag{4.2}$$

η 读作 eta，是**学习率**。学习率控制一次更新移动多远，不能随便设得很大。为什么公式是减号？因为正导数需要往左走，负导数需要往右走，减去导数恰好同时满足这两点。

4.3　完整手算四次迭代

取初值 $x_0=1$，学习率 $\eta=0.2$。第一次先算导数：

$$f'(1) = 2(1-3) = -4.$$

代入更新公式：

$$x_1 = x_0 - \eta f'(x_0) = 1 - 0.2(-4) = \boxed{1.8}.$$

更新前 $f(1)=4$；更新后 $f(1.8)=1.44$，确实下降了。继续照此计算：

迭代次数	当前 x	函数值 $f(x)$
0	1	4
1	1.8	1.44
2	2.28	0.5184
3	2.568	0.186624
4	2.7408	0.06718464

你可以看到 x 从 1 逐渐靠近 3，函数值越来越小。

4.4　为什么“导数”会变成“梯度”？

刚才只有一个变量 x。线性回归有两个参数 w, b，损失函数是 $J(w,b)$。我们分别求偏导

$$\frac{\partial J}{\partial w}, \qquad \frac{\partial J}{\partial b}.$$

把这两个数放进一个列向量，就是**梯度**：

$$\boxed{\nabla J(w,b) = \begin{pmatrix} \partial J/\partial w \\ \partial J/\partial b \end{pmatrix}.} \tag{4.3}$$

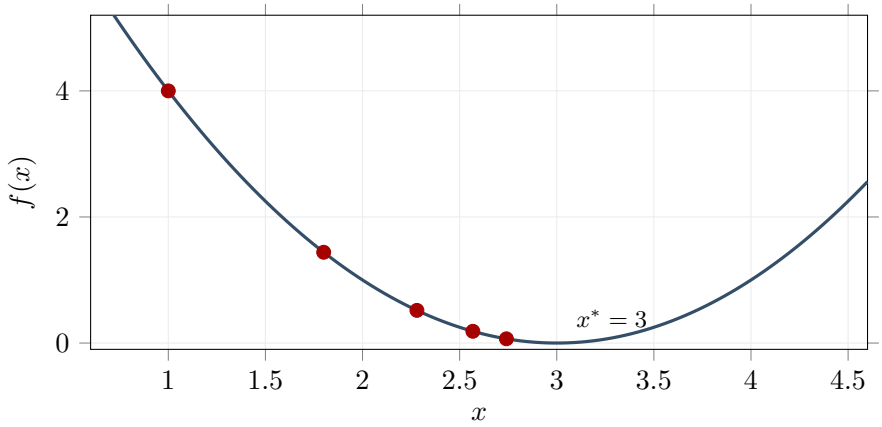


图 1: 从 $x_0 = 1$ 出发，学习率取 0.2 的几次梯度下降迭代。

记号 ∇ 读作 nabla（倒三角）。梯度并不是多了神秘的新计算：它只是把各个参数的偏导数组合起来。

因此，同时更新两个参数的规则是

$$\boxed{\begin{aligned} w_{\text{新}} &= w_{\text{旧}} - \eta \frac{\partial J}{\partial w}, \\ b_{\text{新}} &= b_{\text{旧}} - \eta \frac{\partial J}{\partial b}. \end{aligned}} \tag{4.4}$$

重要：两个偏导都在更新前的同一组旧参数上计算，然后一起更新。不要先改 w 再用新的 w 去计算这一步的 b。

4.5 线性回归：从损失函数求出两个偏导

回顾第二课使用的函数

$$J(w,b) = \frac{1}{2m}\sum_{i=1}^{m}(wx_i + b - y_i)^2.$$

利用复合函数求导法则：

$$\boxed{\begin{aligned} \frac{\partial J}{\partial w} &= \frac{1}{m}\sum_{i=1}^{m}(wx_i + b - y_i)x_i, \\ \frac{\partial J}{\partial b} &= \frac{1}{m}\sum_{i=1}^{m}(wx_i + b - y_i). \end{aligned}} \tag{4.5}$$

读这两个式子时，先看括号中的 $wx_i + b - y_i$：它就是预测值减真实值。对 w 求偏导时，还要乘 x_i；对 b 求偏导时不需要乘。然后对所有样本取平均。

4.6 手工训练一个两样本模型

这一次只用两组数据：

$$(x_1, y_1) = (0, 1), \qquad (x_2, y_2) = (1, 3).$$

线性模型 $\hat{y} = wx + b$ 的损失函数为

$$J(w,b) = \frac{1}{4}[(b-1)^2 + (w+b-3)^2]. \tag{4.6}$$

为什么是这两个平方项？$x=0$ 时预测 b，残差为 $b-1$；$x=1$ 时预测 $w+b$，残差为 $w+b-3$。样本数 $m=2$，所以 $2m=4$。

直接求导：

$$\frac{\partial J}{\partial w}=\frac{1}{2}(w+b-3),\qquad \frac{\partial J}{\partial b}=\frac{1}{2}[(b-1)+(w+b-3)].$$

取初值 $w_0=b_0=0$，学习率 $\eta=0.2$。先在旧参数处计算：

$$\left.\frac{\partial J}{\partial w}\right|_{(0,0)}=-1.5,\qquad \left.\frac{\partial J}{\partial b}\right|_{(0,0)}=-2.$$

再一起更新：

$$w_1=0-0.2(-1.5)=0.3,$$
$$b_1=0-0.2(-2)=0.4.$$

第一次更新后的模型便是

$$\boxed{\hat{y}=0.3x+0.4.} \tag{4.7}$$

检查一下：更新前 $J(0,0)=(1+9)/4=2.5$；更新后

$$J(0.3,0.4)=\frac{(-0.6)^2+(-2.3)^2}{4}=1.4125.$$

这一次更新真的使损失下降了。继续计算偏导并更新，参数会逐渐接近 $w=2,b=1$，最终直线为 $\hat{y}=2x+1$。

4.7 怎么理解学习率？

学习率太小，每次只走一点，训练可能很慢。学习率太大，可能从最小值一侧跳到另一侧，甚至越走越远。**负梯度提供局部下降方向，并不保证使用任何学习率都能下降**。

还可以对刚才的一元二次函数做一个简单检查：

$$x_{k+1}-3=(1-2\eta)(x_k-3).$$

所以固定学习率时，要让误差对任意初值逐渐趋于零，需要 $|1-2\eta|<1$，即 $0<\eta<1$。**这只是对 $f(x)=(x-3)^2$ 的结论，不是所有函数的通用学习率范围**。

4.8 选读：一般参数的统一记号

当参数很多时，把它们记成向量 $\boldsymbol{\theta}$，就可以将所有偏导同时更新写作

$$\boxed{\boldsymbol{\theta}_{k+1}=\boldsymbol{\theta}_k-\eta_k\nabla J(\boldsymbol{\theta}_k).}$$

如果 J 是凸可微函数，一个梯度为零的点是全局极小点；但一般非凸函数并没有这样的保证。关于高维步长、曲率、收敛速度的更严格结论属于进阶优化理论，本课不把它们当作入门要求。

4.9 这一课应当掌握什么

1. 会判断一元函数导数的正负对应怎样的更新方向。
2. 能根据 $x_{\text{新}}=x_{\text{旧}}-\eta f'(x_{\text{旧}})$ 手算一次迭代。

3. 知道梯度是偏导数组成的向量，并能同时更新 w, b。
4. 理解“先预测、再计算误差、再求偏导、最后更新参数”的流程。
5. 知道学习率影响迭代过程，不能只看导数符号。

自检： $f(x) = (x-4)^2$，初值 $x_0 = 2$，学习率 $\eta = 0.1$。第一次更新得到多少？

本课的迭代思想与基础最优化教材中的梯度法一致[4]。

5 第四课：逻辑回归与二分类

5.1 成绩是数值，“能否通过”却是类别

第二课用 $\hat{y} = wx + b$ 预测数值。如果现在改成预测学生能否通过考试，真实标记只有两种：

$$y = \begin{cases} 1, & \text{通过考试,} \\ 0, & \text{未通过考试.} \end{cases}$$

例如某个教学数据集如下：

学习时间 x（小时）	是否通过 y
1	0
2	0
3	0
4	1
5	1

我们希望估计的是“通过的概率”。但直接把直线当作概率会出问题：假设线性预测为 $\hat{y} = 0.3x - 0.4$，则 $x = 1$ 时得到 -0.1，$x = 6$ 时得到 1.4。概率却必须在 0 到 1 之间。

因此，逻辑回归并不是直接把线性输出作为概率，而是**先计算线性得分，再用另一个函数把它映射到** $(0, 1)$。

5.2 Sigmoid 函数怎么把实数变成概率？

定义 Logistic（常称 Sigmoid）函数：

$$\boxed{\sigma(z) = \frac{1}{1 + e^{-z}}, \qquad z \in \mathbb{R}.} \tag{5.1}$$

e 是自然对数的底数。我们不需要一开始就背下曲线，只需代几个数：

$$\sigma(0) = 0.5, \quad \sigma(2) \approx 0.8808, \quad \sigma(-2) \approx 0.1192.$$

z 越大，输出越接近 1；z 越小，越接近 0；任何有限 z 的输出都严格在两者之间。

5.3 把线性模型代进去，就得到逻辑回归

设 $z = wx + b$，用 $\hat{p}$ 表示预测“通过”的概率：

$$\boxed{\hat{p} = P(Y = 1 \mid X = x) = \sigma(wx + b) = \frac{1}{1 + e^{-(wx+b)}}.} \tag{5.2}$$

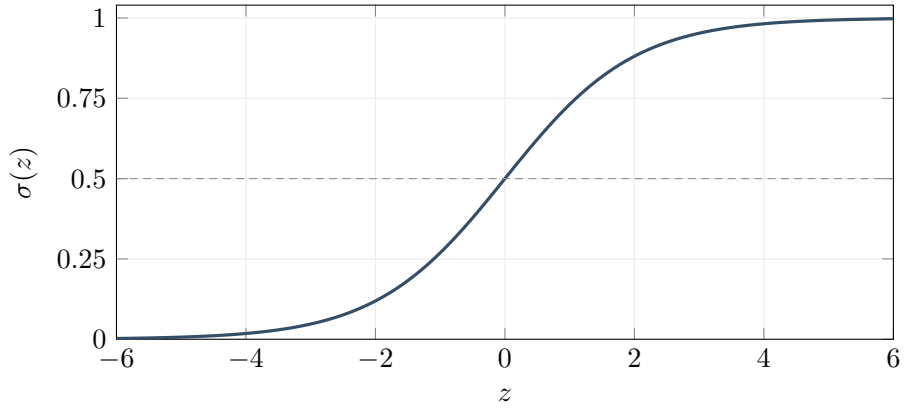


图 2: Sigmoid 函数：任意实数输入，输出一个介于 0 和 1 之间的数。

例如 $w = 1, b = 0, x = 2$，则 $\hat{p} = \sigma(2) \approx 0.8808$，可解读为模型预测通过概率约 88.08%。是否最终输出类别 1，还需要指定阈值；常见做法是概率不小于 0.5 时预测类别 1。

5.4 为什么叫“对数几率回归”？（理解即可）

将概率简记为 $p = \sigma(wx + b)$，那么

$$p = \frac{e^{wx+b}}{1 + e^{wx+b}}, \qquad 1 - p = \frac{1}{1 + e^{wx+b}}.$$

两式相除，再取自然对数：

$$\frac{p}{1-p} = e^{wx+b}, \qquad \boxed{\ln \frac{p}{1-p} = wx + b.}$$

这便是对数几率（log-odds）与线性特征的关系：

$$\log \frac{p}{1-p} = wx + b. \tag{5.3}$$

因此逻辑回归对**概率的对数几率**做线性建模，而不是要求概率本身随 x 线性变化[1]第 3 章。

5.5 概率有了，参数怎么训练？

对于第 i 个样本，真实标签 y_i 只能是 0 或 1，预测正类概率为 $p_i = \sigma(wx_i + b)$。那么

$$P(Y_i = 1 \mid x_i) = p_i, \qquad P(Y_i = 0 \mid x_i) = 1 - p_i.$$

两种情况可合并为伯努利分布的概率质量函数：

$$P(y_i \mid x_i) = p_i^{y_i}(1 - p_i)^{1-y_i}. \tag{5.4}$$

看一下公式为什么成立：若 $y_i = 1$，右边为 p_i；若 $y_i = 0$，右边为 $1 - p_i$。

在给定特征后各样本类别条件独立的假设下，整批已观测标签的条件似然是

$$\mathcal{L}(w, b) = \prod_{i=1}^{m} p_i^{y_i}(1 - p_i)^{1-y_i}. \tag{5.5}$$

极大似然估计希望找到 w, b，使观测到这些标签的条件概率尽可能大。为方便求导，先取对数，把乘积变成求和：

$$\ln \mathcal{L}(w,b) = \sum_{i=1}^{m} \big[y_i \ln p_i + (1-y_i)\ln(1-p_i) \big].$$

再乘负号并取平均，极大化便转化为熟悉的极小化：

$$\boxed{J(w,b) = -\frac{1}{m}\sum_{i=1}^{m} \left[y_i \ln p_i + (1-y_i)\ln(1-p_i) \right].} \tag{5.6}$$

它叫作二**元交叉熵**（Binary Cross-Entropy，BCE），也是当前伯努利概率模型的平均负对数似然。

5.6 交叉熵为什么合理？先看几个数

单个样本的交叉熵是

$$\ell = -y \ln p - (1-y)\ln(1-p).$$

真实类别 y	预测正类概率 p	单样本损失 ℓ
1	0.9	0.105
1	0.1	2.303
0	0.1	0.105
0	0.9	2.303

如果真实类别是 1，预测概率 0.9 比 0.1 好得多；相反，如果真实类别是 0，预测 $p = 0.9$ 就意味着模型非常自信地预测错了，因此损失很大。这也是交叉熵相比只看“分类对不对”包含更多信息的原因。

5.7 从求导得到梯度更新公式

前一课已经学过梯度下降，现在我们只需对 J 求偏导。第一步，先推导 Sigmoid 的导数：

$$\sigma'(z) = \frac{e^{-z}}{(1+e^{-z})^2} \tag{5.7}$$

$$= \boxed{\sigma(z)\big[1-\sigma(z)\big].} \tag{5.8}$$

第二步，对单样本交叉熵 $\ell = -y\ln p - (1-y)\ln(1-p)$ 关于 p 求导：

$$\frac{\partial \ell}{\partial p} = -\frac{y}{p} + \frac{1-y}{1-p}.$$

因为 $p = \sigma(z)$，链式法则给出

$$\begin{aligned}
\frac{\partial \ell}{\partial z} &= \frac{\partial \ell}{\partial p}\frac{\mathrm{d}p}{\mathrm{d}z} \\
&= \left(-\frac{y}{p} + \frac{1-y}{1-p} \right) p(1-p) \\
&= -y(1-p) + (1-y)p \\
&= \boxed{p - y}.
\end{aligned}$$

这一步最值得记住：**对线性得分 z 求导，得到预测概率减去真实标签**。

第三步，由 $z = wx + b$ 可知 $\partial z/\partial w = x$、$\partial z/\partial b = 1$。所以对所有样本取平均：

$$\boxed{\begin{aligned}\frac{\partial J}{\partial w} &= \frac{1}{m}\sum_{i=1}^{m}(p_i - y_i)x_i,\\ \frac{\partial J}{\partial b} &= \frac{1}{m}\sum_{i=1}^{m}(p_i - y_i).\end{aligned}} \tag{5.9}$$

对应的梯度下降更新为

$$\begin{aligned}w_{\text{新}} &= w_{\text{旧}} - \eta\frac{1}{m}\sum_{i=1}^{m}(p_i - y_i)x_i,\\ b_{\text{新}} &= b_{\text{旧}} - \eta\frac{1}{m}\sum_{i=1}^{m}(p_i - y_i).\end{aligned} \tag{5.10}$$

与线性回归很像：先算预测与真实值的差，再求平均并更新参数。只是逻辑回归用的是概率 p_i，线性回归用的是线性预测值 $\hat{y}_i$。

5.8 手算一次逻辑回归训练

为了计算简单，只用一个样本：$x = 2, y = 1$，并取

$$w_0 = 0, \quad b_0 = 0, \quad \eta = 0.1.$$

第一步，预测概率：

$$p_0 = \sigma(0 \times 2 + 0) = 0.5.$$

第二步，计算两个偏导：

$$\frac{\partial J}{\partial w} = (0.5 - 1) \times 2 = -1, \qquad \frac{\partial J}{\partial b} = 0.5 - 1 = -0.5.$$

第三步，更新：

$$w_1 = 0 - 0.1(-1) = 0.1, \qquad b_1 = 0 - 0.1(-0.5) = 0.05.$$

第四步，再预测：

$$p_1 = \sigma(0.1 \times 2 + 0.05) = \sigma(0.25) \approx 0.5622.$$

因为真实标签是 1，正类概率从 50% 上升到约 56.22%，这一步朝正确方向变化。单样本交叉熵也由约 0.6931 降到约 0.5759。**这只验证了当前样本的当前更新，并非任意学习率都保证下降**。

5.9 选读：凸性、可分性与边界条件

对于标准的线性逻辑回归，未正则化的平均负对数似然关于模型参数是凸函数；这不意味着 Sigmoid 曲线本身全局凸。凸性也不保证任意学习率都收敛，更不保证在每种数据集上都存在唯一的有限最优参数。例如数据线性完全可分时，未加正则化的参数极大似然估计可能没有有限解。这些是进一步学习优化理论时需要考虑的边界，本课只需知道它们存在。

5.10 这一课应当掌握什么

1. 知道二分类标签 $y \in \{0, 1\}$ 与连续数值预测不同。
2. 会根据 $z = wx + b$ 和 Sigmoid 计算预测概率。

3. 知道交叉熵可以从伯努利似然推导，不是随意挑选的函数。
4. 会由 $\sigma'(z)=\sigma(z)[1-\sigma(z)]$ 推得 $\partial \ell/\partial z = p-y$。
5. 能够像上一课一样手算一次对 w, b 的同步更新。

自检：若真实标签为 1，预测概率由 0.1 改成 0.9，交叉熵如何变化？为什么逻辑回归的线性部分不能直接解释成概率？

本课内容对应周志华《机器学习》的对数几率回归与 ISLP 分类章节[1,3]。

6 第五课：过拟合、泛化与模型评价

6.1 训练数据全预测对了，模型就是好模型吗？

先看一个最容易理解的例子。三名学生的教学数据是：

学生	每天学习时间	成绩
A	1 小时	60
B	2 小时	70
C	3 小时	80

有两个模型都能把这三组训练数据预测正确。

模型 A：根据线性规律预测，$f_A(x)=50+10x$。

模型 B：把“1 小时对应 60、2 小时对应 70、3 小时对应 80”直接记住；遇到其他学习时间统一预测 60。

两者在训练集上的 MSE 都为 0。现在来了没有参加训练的学生 D，学习时间 4 小时，假设真实成绩是 90。模型 A 预测 90，平方损失为 0；模型 B 预测 60，平方损失为 $(90-60)^2=900$。

比较项目	模型 A	模型 B
训练集 MSE	0	0
新样本预测	90	60
新样本平方损失	0	900

所以，**训练误差一样小，不代表新数据上的效果一样好**。这个例子完全是人工构造的，不表示真实成绩一定遵循线性规律。

6.2 什么是过拟合和欠拟合？

过拟合（Overfitting）指模型过度适应训练样本的细节，甚至记住其中的噪声和偶然性，结果在未见数据上表现不佳。刚才的记忆模型就是直观示例。

欠拟合（Underfitting）则是模型能力或训练程度不足，没能充分表示数据中的主要规律。例如数据明显呈曲线趋势，却始终只用一条直线拟合，训练误差可能很大，新数据预测也不理想。

情况	训练数据表现	未见数据表现
欠拟合	常常不好	常常不好
合理拟合	能学到主要规律	相对稳定
过拟合	可能特别好	可能明显变差

模型更复杂并非一定过拟合，模型更简单也并非一定欠拟合。关键是看它是否适应真实数据规律，以及是否能推广到新数据。

6.3 训练误差、测试误差、泛化误差有什么区别？

训练误差在用来训练模型的样本上计算。以平方损失为例：

$$\mathrm{MSE}_{\text{train}} = \frac{1}{m}\sum_{i=1}^{m}(y_i - \hat{f}(x_i))^2. \tag{6.1}$$

测试误差在未参与训练的数据上计算。如果测试集有 n 个样本，就写为

$$\mathrm{MSE}_{\text{test}} = \frac{1}{n}\sum_{i=1}^{n}(y_i' - \hat{f}(x_i'))^2.$$

这里的撇号只是提示“它们来自另一批测试数据”。

泛化误差关心模型对目标数据分布中的新观测的平均损失。假设训练得到的 $\hat{f}$ 已经确定，定义

$$R(\hat{f}) = \mathbb{E}_{(X,Y)\sim P}[L(Y, \hat{f}(X))]. \tag{6.2}$$

$\mathbb{E}$ 是期望，P 是目标数据的联合分布。我们通常不知道这个分布，所以难以直接计算 $R(\hat{f})$；独立测试集上的平均损失只是它的**统计估计**。

名称	在哪些数据上计算?	能告诉我们什么?
训练误差	训练时见过的样本	拟合这些样本的程度
测试误差	训练与选择时未使用的样本	对新数据表现的有限样本估计
泛化风险	目标数据的概率分布	理论上的平均预测损失

6.4 如何划分训练集、验证集和测试集？

训练集用于确定模型参数，例如线性回归中的 w, b。**验证集**用于比较模型或选择超参数，例如选择多项式次数、学习率。**测试集**在模型选择完成后用于最终评估，尽量不要反复用于调参。

假设有 10 000 条样本，一种示例划分是训练集 7 000 条、验证集 1 500 条、测试集 1 500 条（70%/15%/15%）。这不是固定标准：样本量、时间顺序、是否有重复测量等都会影响合理划分。

为什么不能在测试集上反复挑选模型？因为它虽然没有参与梯度计算，却已经参与了模型选择，最后的测试成绩可能因此偏于乐观。测试集最重要的价值是**保持独立**。

6.5 留出法与五折交叉验证

留出法（Hold-out）是把数据拆成相互不重叠的训练部分与评价部分，例如 700 条训练、300 条测试。不同随机划分可能带来不同结果。

K 折交叉验证（K-Fold Cross Validation）利用多次划分缓解偶然性。以 5 折为例：

1. 将训练部分划分为五份，记作第 1 ~ 5 折。
2. 第一次用第 2 ~ 5 折训练，用第 1 折验证。
3. 第二次用第 1、3、4、5 折训练，用第 2 折验证。
4. 依次轮换，使每一折都恰好作为一次验证集。
5. 把五次验证误差汇总，用来比较模型。

当各折样本量相同时，交叉验证平均误差为

$$\boxed{E_{\mathrm{CV}} = \frac{1}{K}\sum_{k=1}^{K} E_k.} \tag{6.3}$$

E_k 表示第 k 次验证的平均误差。若各折样本数 n_k 不同，若希望求所有折外预测的样本平均，则使用加权式

$$E_{\mathrm{CV,weighted}} = \frac{\sum_k n_k E_k}{\sum_k n_k}. \tag{6.4}$$

五折、十折都很常见，但交叉验证不是无条件“越多折越好”。时间序列、同一学生多次记录等具有依赖关系的数据，还需要符合数据结构的拆分方式。

6.6 容易忽略的数据泄漏

假设准备把特征标准化：

$$z = \frac{x - \mu}{\sigma}.$$

若先用全部数据计算均值 μ 和标准差 σ，然后才划分训练与测试，那么测试集的信息已经提前进入训练流程，这叫作**数据泄漏**（Data Leakage）。

正确的基本步骤是：只在训练数据上学习预处理参数，再用这些参数转换验证集和测试集。交叉验证时也必须在每一折的训练部分重新拟合标准化器。数据泄漏可能让评价结果看起来比实际部署效果更好。

6.7 偏差与方差：先理解含义，再看公式

为什么增加模型复杂度通常能降低训练误差，却不一定改善测试效果？一个常用的分析视角是**偏差—方差权衡**。

偏差（Bias）描述模型的平均预测相对于真实规律的系统性偏离。例如真实规律是一条弯曲的曲线，却坚持用直线去拟合，就可能存在较大偏差。

方差（Variance）描述模型对训练数据变化的敏感程度。换一批训练样本，模型预测就发生很大变化，通常意味着较高的方差。

不可约误差则来自观测中的随机噪声，即使知道真实规律也未必能把每个样本预测到完全准确。

在适当条件下，例如固定输入 x、$Y = f(x) + \varepsilon$、噪声均值为零、方差为 σ^2 且与训练过程独立，平方预测误差可以分解为

$$\mathbb{E}[(Y - \hat{f}_D(x))^2] = \mathrm{Bias}(\hat{f}_D(x))^2 + \mathrm{Var}(\hat{f}_D(x)) + \sigma^2. \tag{6.5}$$

这里的期望要同时对重复抽取的训练数据与新的观测噪声考虑。**初学时只需理解三项各代表什么，不能把这条式子无条件套用到准确率、交叉熵等指标**。模型灵活性与偏差、方差的关系是常见趋势，而不是所有模型的单调定理[3]第 2 章。

6.8 回归模型怎么评价？

最常用的指标之一仍是 MSE：

$$\mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2.$$

例如真实值为 $(10, 20, 30)$，预测为 $(11, 18, 33)$，平方误差就是 $1, 4, 9$，所以 $\mathrm{MSE} = (1+4+9)/3 = 14/3$。比较模型时，必须说明是在训练集、验证集还是测试集上算的 MSE。

6.9 分类模型怎么评价？先读懂混淆矩阵

分类模型可以使用**准确率**（Accuracy）：

$$\mathrm{Accuracy} = \frac{\text{预测正确的样本数}}{\text{总样本数}}.$$

但准确率不总够用。假设 100 个样本只有 5 个正类，全部预测为负类也有 95% 准确率，却一个正类也没有找出。

二分类中常用四个缩写：

- TP：实际为正，预测也为正（真正例）；
- FP：实际为负，误报为正（假正例）；
- TN：实际为负，预测也为负（真反例）；
- FN：实际为正，漏报为负（假反例）。

表 2: 混淆矩阵的一个教学算例

	预测正类	预测负类
实际正类	$TP = 24$	$FN = 16$
实际负类	$FP = 6$	$TN = 54$

查准率（Precision）回答“预测为正的样本中，有多少是真的正？”：

$$\mathrm{Precision} = \frac{TP}{TP + FP} = \frac{24}{24 + 6} = 80\%.$$

查全率（Recall）回答“真正的正类中，有多少被找到了？”：

$$\mathrm{Recall} = \frac{TP}{TP + FN} = \frac{24}{24 + 16} = 60\%.$$

如果希望同时考虑两者，常用它们的调和平均 F_1：

$$\boxed{F_1 = \frac{2PR}{P + R} = \frac{2 \times 0.8 \times 0.6}{0.8 + 0.6} \approx 0.686.}$$

在这条式子中，P 和 R 分别是 Precision 与 Recall，不是概率分布符号。当分母为零时，需要另行规定指标处理规则。

ROC 曲线通过改变分类阈值，绘制真正例率 $TPR = TP/(TP + FN)$ 与假正例率 $FPR = FP/(FP + TN)$ 的关系；AUC 是曲线下面积，常用于衡量排序能力。它们比一个固定阈值的准确率提供不同的信息。

6.10 一次多项式实验为什么有帮助？

考虑构造数据 $y = \sin x + \varepsilon$，其中 ε 是均值为零的随机扰动。用不同次数的多项式拟合，教学记录中曾得到以下一组**固定数据划分下的示例结果**：这组数值仅说明这一教学设置下的现象，不

表 3: 不同模型复杂度下的一次教学实验记录

多项式次数	训练 MSE	测试 MSE	观察
1	0.1869	0.1740	模型较简单，误差偏高
8	0.0368	0.0359	本次训练和测试误差均较低
25	0.0269	0.2600	训练更好，测试明显变差

是所有数据都满足的规律。正式实验需保存生成数据的随机种子、划分、预处理与运行环境，才能独立复现。比较多个候选模型时，应先通过验证集或交叉验证选择，最终测试集留作独立评估。

6.11 这一课应当掌握什么

1. 训练误差小不保证泛化误差小；测试误差只是泛化风险的估计。
2. 理解过拟合与欠拟合，并能用学生成绩例子解释区别。
3. 分清训练集、验证集、测试集的角色和数据泄漏风险。
4. 能口头复述五折交叉验证，并会计算平均验证误差。
5. 能从混淆矩阵手算 Precision、Recall 与 F_1。

自检：为什么“先对全体样本做标准化，再分训练集与测试集”可能导致不真实的高分？为什么训练误差从 0.0368 降到 0.0269 并不意味着模型变好？

主要参考周志华《机器学习》第 2 章、李航《统计学习方法》模型评估部分及 ISLP 第 2 章[1-3]。

7 动手验证：把公式翻译成 Python

前面五课都是先通过手算理解，再逐渐引入通用记号。现在使用 Python 把一些计算重复很多次，并核对结果。所有示例都可以从仓库根目录运行。先执行 `python -m pip install -r requirements.txt` 安装 NumPy。

7.1 实验一：用网格搜索找 w

第一课的三个训练样本是 $(1,2),(2,4),(3,6)$，模型为 $\hat{y}=wx$。我们知道 $w=2$ 时 MSE 为零。现在不直接用解析公式，而是让计算机枚举 0 到 4 之间的一些候选参数，每次计算

$$J(w)=\frac{1}{3}\left[(w-2)^2+(2w-4)^2+(3w-6)^2\right].$$

代码保存在 `code/grid_search_intro.py`：

Listing 1: 从损失函数出发搜索最佳参数

```python
"""Lesson 1: learn a one-parameter model by trying candidate values."""
import numpy as np

x = np.array([1.0, 2.0, 3.0])
y = np.array([2.0, 4.0, 6.0])

candidates = np.linspace(0.0, 4.0, 401)
losses = []
for w in candidates:
    prediction = w * x
    mse = np.mean((prediction - y) ** 2)
    losses.append(mse)

best_index = int(np.argmin(losses))
best_w = float(candidates[best_index])
best_mse = float(losses[best_index])
new_prediction = best_w * 4.0

print(f"best w = {best_w:.1f}")
print(f"training MSE = {best_mse:.1f}")
print(f"prediction when x=4 = {new_prediction:.1f}")
assert np.isclose(best_w, 2.0)
assert np.isclose(best_mse, 0.0)
assert np.isclose(new_prediction, 8.0)
```

它会输出最优候选参数 2.0、最小训练 MSE 为 0.0，以及 $x=4$ 时的预测 8.0。网格搜索只在*列举过的参数*中找最优值；本例恰好包含真实最优参数，并不意味着一般情形也会找到精确最优解。

7.2 实验二：最小二乘公式与两次求偏导

第二课使用四组数据 $(1,2),(2,3),(3,5),(4,4)$。先求 $\bar{x},\bar{y}$，再计算

$$\hat{w}=\frac{\sum_i (x_i-\bar{x})(y_i-\bar{y})}{\sum_i (x_i-\bar{x})^2},\qquad \hat{b}=\bar{y}-\hat{w}\bar{x}.$$

结果应该是 $\hat{w}=0.8,\hat{b}=1.5$，训练 MSE 为 0.45。这次不必依赖封装好的回归库：把数学公式直接写成 `np.mean` 与 `np.sum`，可以看到每一步对应哪条公式。示例代码：

Listing 2: 直接将最小二乘公式翻译为 NumPy

```python
"""Lesson 2: turn the ordinary least squares formulas into code."""
import numpy as np

```

```python
x = np.array([1.0, 2.0, 3.0, 4.0])
y = np.array([2.0, 3.0, 5.0, 4.0])

x_mean = float(np.mean(x))
y_mean = float(np.mean(y))
numerator = float(np.sum((x - x_mean) * (y - y_mean)))
denominator = float(np.sum((x - x_mean) ** 2))
assert denominator > 0.0, "Cannot identify slope when all x are equal."

w = numerator / denominator
b = y_mean - w * x_mean
prediction = w * x + b
mse = float(np.mean((prediction - y) ** 2))

print(f"mean x = {x_mean:.1f}, mean y = {y_mean:.1f}")
print(f"w = {w:.1f}, b = {b:.1f}")
print(f"training MSE = {mse:.2f}")
print(f"prediction when x=5 = {w * 5.0 + b:.1f}")
assert np.isclose(w, 0.8)
assert np.isclose(b, 1.5)
assert np.isclose(mse, 0.45)
```

仓库另外保留 `code/linear_regression_demo.py`，用于比较最小二乘数值解与梯度下降结果。原有脚本使用另一组数据 $(1,2),(2,3),(3,5)$：这组数据的斜率是 $3/2$、截距是 $1/3$，不是刚才四样本例子的 0.8、1.5。**不要混用不同实验的数据和答案**。

7.3 实验三：反复迭代就是梯度下降

第三课已经手算过

$$f(x) = (x-3)^2, \qquad x_0 = 1, \qquad \eta = 0.2.$$

第一次更新得到 $x_1 = 1.8$，之后是 2.28、2.568 等。使用一个 `for` 循环，把“求导 → 更新 → 计算新函数值”重复即可：

Listing 3: 把一元函数梯度下降写成循环

```python
"""Lesson 3: trace gradient descent on f(x)=(x-3)^2."""

def f(x):
    return (x - 3.0) ** 2


def derivative(x):
    return 2.0 * (x - 3.0)


x = 1.0
learning_rate = 0.2
print("step | current x | f(x)")
for step in range(5):
    print(f"{step:4d} | {x:9.4f} | {f(x):.8f}")
    x = x - learning_rate * derivative(x)

# After updates at steps 0..4, the next point is x=2.84448.
assert abs(x - 2.84448) < 1e-10
```

在更复杂的线性回归里，循环内部只需改为同时更新 w, b。关键是先从旧参数处计算两个偏导，再一起更新。软件重复操作很快，但理解它的最好方式仍是手算第一步。

7.4 实验四：逻辑回归的一步参数更新

第四课用一个样本 $x = 2, y = 1$ 演示：

$$w_0 = b_0 = 0, \quad \eta = 0.1, \quad p_0 = \frac{1}{1 + e^0} = 0.5.$$

解析梯度为 $\partial_w J = (p - y)x = -1$ 与 $\partial_b J = p - y = -0.5$。所以一步更新得 $w_1 = 0.1, b_1 = 0.05$，新概率为 $\sigma(0.25) \approx 0.5622$：

Listing 4: 一个样本的逻辑回归梯度更新

```python
"""Lesson 4: one SGD step for a one-sample logistic regression."""
from math import exp, log


def sigmoid(z):
    return 1.0 / (1.0 + exp(-z))


x, y = 2.0, 1.0
w, b = 0.0, 0.0
learning_rate = 0.1

p_before = sigmoid(w * x + b)
dw = (p_before - y) * x
db = p_before - y

# Both gradients are calculated before changing either parameter.
w_new = w - learning_rate * dw
b_new = b - learning_rate * db
p_after = sigmoid(w_new * x + b_new)
loss_before = -log(p_before)
loss_after = -log(p_after)

print(f"before: p={p_before:.4f}, loss={loss_before:.4f}")
print(f"after: w={w_new:.2f}, b={b_new:.2f}")
print(f"after: p={p_after:.4f}, loss={loss_after:.4f}")

assert abs(w_new - 0.1) < 1e-12
assert abs(b_new - 0.05) < 1e-12
assert abs(p_after - 0.5621765) < 1e-6
assert loss_after < loss_before
```

仓库原有的 `code/logistic_gradient_check.py` 则做另一件事：用数值差分近似求导，检查解析梯度的实现是否正确。对某个参数分量 β_j，中心差分近似为

$$\frac{\partial J}{\partial \beta_j} \approx \frac{J(\boldsymbol{\beta} + h\boldsymbol{e}_j) - J(\boldsymbol{\beta} - h\boldsymbol{e}_j)}{2h}. \tag{7.1}$$

这里不要求初学者独立证明该近似，只需要知道它可以帮助发现代码求导错误。

7.5 实验五：训练误差下降为何不代表测试误差下降？

第五课介绍多项式复杂度与过拟合。仓库已有 `code/polynomial_comparison.py`，可直接运行，记录不同次数模型的训练、测试 MSE。在比较结果时，一定看清数据生成方式、训练与测试的划分，以及数值是否来自同一随机种子。

7.6 实验记录应当保存什么？

为了真正复现，一份实验记录至少需要交代：数据或生成规则、样本数量、训练/验证/测试划分、随机种子、模型形式、损失函数、优化方法、学习率或其他超参数、评价指标、Python 与依赖版本。

这部分的主要目标不是堆积代码，而是建立“公式 $\leftrightarrow$ 手工计算 $\leftrightarrow$ Python 结果”的联系。如果输出与手算不一致，优先核对数据是否相同、分母是 n 还是 $2n$、残差方向以及更新时有没有混用新旧参数。

8 选读：下一阶段的学习路线

前面五课的核心是理解一个模型怎样由数据构造、怎样训练、怎样判断预测表现。这一节仅帮助辨认下一阶段可能遇到的概念，不要求立即推导或掌握。

8.1 正则化：除了拟合，还要控制复杂度

仅仅减小训练 MSE，可能会使模型过度追求样本上的小误差。一个常见思路是给目标函数额外增加惩罚项：

$$\min_{\boldsymbol{\theta}} \left\{ \frac{1}{m} \sum_{i=1}^{m} L(y_i, f_{\boldsymbol{\theta}}(x_i)) + \lambda \Omega(\boldsymbol{\theta}) \right\}, \qquad \lambda \geq 0.$$

现在只需这样理解：第一部分衡量拟合误差；第二部分表达对参数大小或结构的额外偏好；λ 控制二者的权衡。更具体的 L1、L2 正则化及其优化性质留到正式学习时再展开。

8.2 从一个变量到很多变量

最初学习 $\hat{y} = wx + b$，是因为它可以手算；更一般的模型可能有很多特征和参数，需要利用向量和矩阵简化书写。先熟悉单变量的偏导数，之后再回到第二课的矩阵正规方程，不必本末倒置。

8.3 从训练模型到理解统计不确定性

线性回归可以给出参数点估计，但真实数据存在随机性。统计学还要讨论估计量的偏差、方差、区间估计及其前提。模型能预测得好与参数是否具有因果解释也不是同一问题。

8.4 阅读前的三个问题

1. 为什么增加一个复杂的模型可能改善训练误差，却使测试误差变大?
2. 对同一批样本，为什么换一个损失函数可能得到不同参数?

3. 如果两个参数都要更新，为什么应先用旧参数计算同一轮的偏导?

如果能回答这些问题，说明前五课的知识已经建立了初步联系。

A 常用符号：先查含义，再读公式

同一套笔记里有时用 m，有时用 n 表示样本数；它们都是“本次数据集有多少行”，不是两个不同的数学概念。为了尊重各课算例的写法，下表同时列出这些记号。

符号	本笔记中的含义
x_i	第 i 个样本的输入（单个特征时为一个数）。
$\boldsymbol{x}_i$	第 i 个样本的多个数值特征组成的向量。
y_i	第 i 个样本的真实结果或标记。
$\hat{y}_i$	模型对第 i 个样本的预测结果。
m, n	样本数量；不同课中可能使用不同字母。
d, p	特征维数或参数维数，具体按公式附近的定义使用。
D	训练数据集。
w, b	一元线性或逻辑回归的斜率（权重）、截距（偏置）。
$\bar{x}, \bar{y}$	输入和目标的样本均值。
e_i	残差；不同章节需留意预测减真实、或真实减预测的约定。
L	单个样本的损失函数。
RSS	残差平方和。
MSE	均方误差，平方误差的样本平均。
J	训练时要减小的目标函数；有时为 MSE 的一半。
$\mathcal{F}$	候选模型构成的集合（模型族）。
$\widehat{R}_D(f)$	模型在训练样本上的平均损失（经验风险）。
$R(f)$	在目标数据分布下的期望损失（总体风险）。
η	梯度下降的学习率。
∇J	把各参数的偏导数组合成的梯度向量。
$\sigma(z)$	Sigmoid 函数，$\sigma(z) = 1/(1+e^{-z})$。
p_i	逻辑回归对第 i 个样本预测的正类概率。
TP, FP, TN, FN	真正例、假正例、真反例、假反例。
K	K 折交叉验证的折数。
$\mathbb{E}$	数学期望，即按概率分布取平均。

遇到陌生公式时，先找出“数据在哪里、参数在哪里、预测在哪里、损失在哪里”。对于一般集合的条件记号，本笔记统一采用 $\mid$，例如 $\{w \in \mathbb{R} \mid w > 0\}$。

B 修订记录

日期	版本	修订内容
2026-11-11	v0.3	根据前五课教学记录重构正文：增加概念解释、逐步求导、手算表格、原有教学算例与可运行 Python 脚本；保留必要的统计条件，将较难的矩阵表达式移至选读部分。
2026-11	v0.1	建立 LaTeX 模块化笔记、基础统计学习内容和数值实验。

参考文献

[1] 周志华. 机器学习[M]. 清华大学出版社, 2016.

[2] 李航. 统计学习方法[M]. 2 版. 清华大学出版社, 2019.

[3] James G, Witten D, Hastie T, Tibshirani R, Taylor J. An Introduction to Statistical Learning: With Applications in Python[M/OL]. Springer, 2023. DOI: 10.1007/978-3-031-38747-0.

[4] 刘浩洋, 户将, 李勇锋, 文再文. 最优化计算方法（简化版）[M]. 2020.